# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

# Mobile Programming

(IT4785/IT4785E)




ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

# Lesson 6: Display lists and use Material Design (part 1)

ONE LOVE. ONE FUTURE.

# Goals

## Kotlin fundamentals

- Functional programming
- Generics variance
- Collection internals
- Advanced design patterns

## Composables

- Scrollable list
- App icon
- Grid
- Material design, theming
- Animation
- Accessibility
- Practice with real app: Superheroes

HUST

hust.edu.vn fb.com/dhbkhn

# PATHWAY 1 - More Kotlin fundamentals

- Generics, objects, and extensions
- Collections in Kotlin
- Higher-order functions with collections

HUST

hust.edu.vn fb.com/dhbkhn

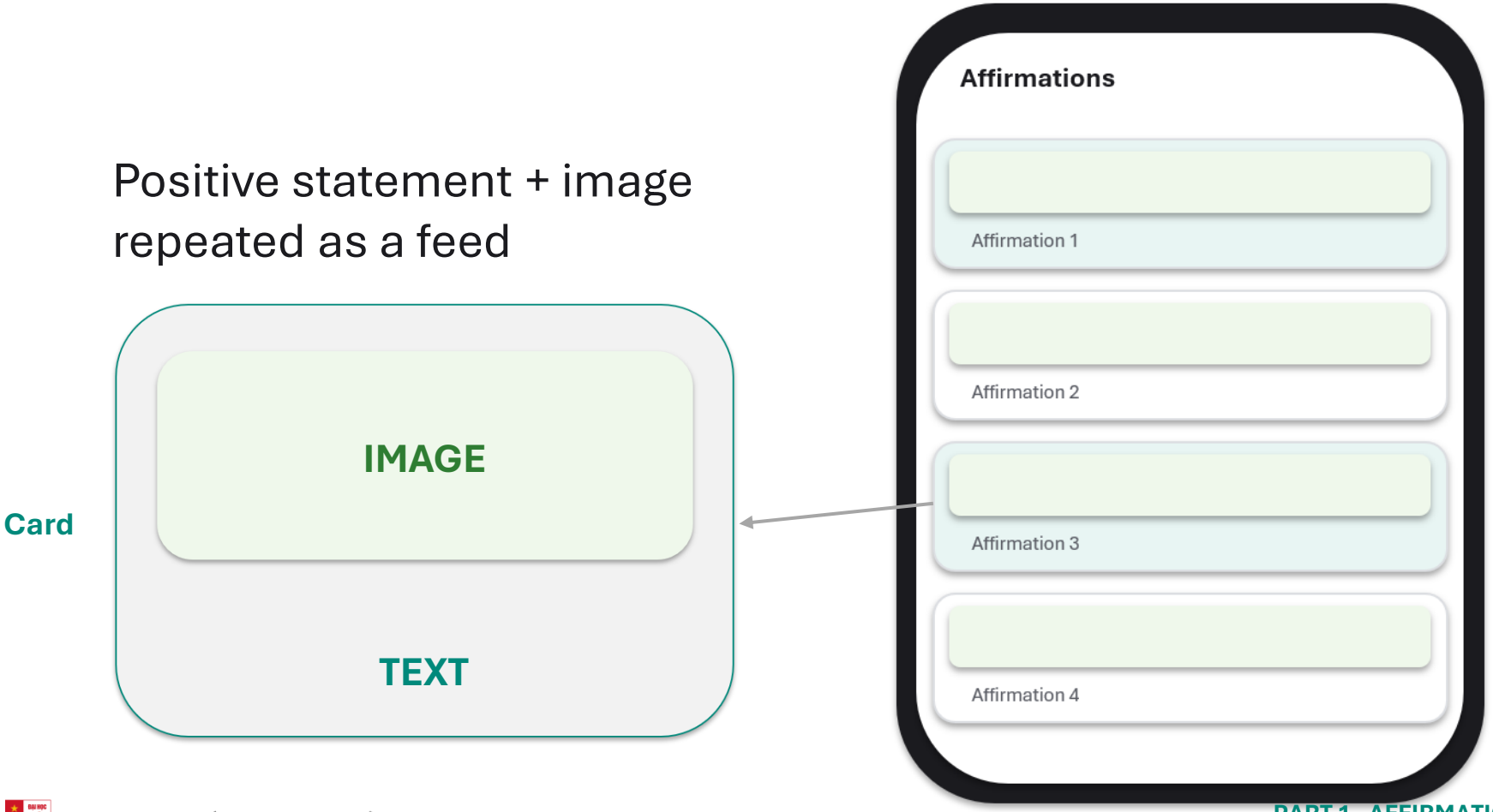
# PATHWAY 2 - Build a scrollable list

- Affirmations app
- Scrollable list
- App icon
- Build a grid

# What Is the Affirmations App?

- The app shows a scrollable feed of positive statements, each paired with an image

Positive statement + image repeated as a feed



# Problem: One Card Isn't a List

- We can hand-write one Card, but not many

**1 CARD**

hand code ✓

→

**50 CARDS**

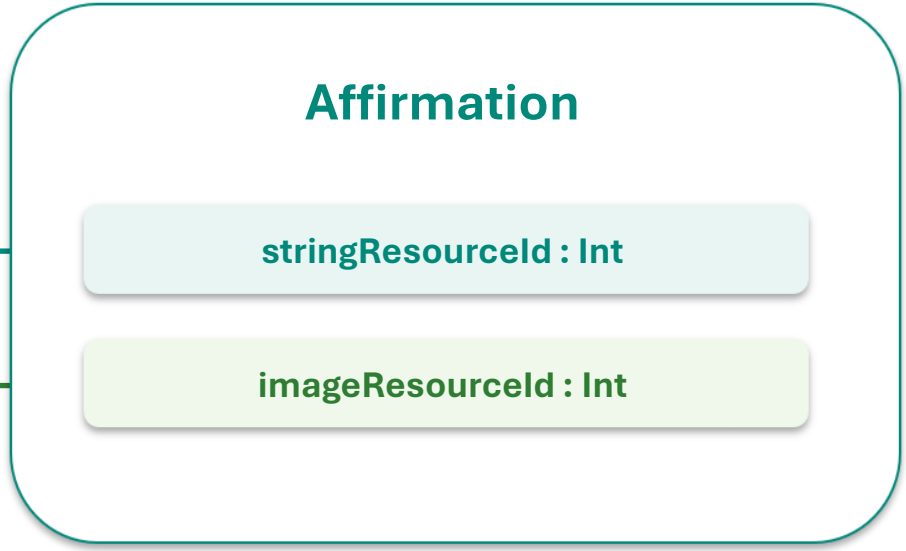
**loop over data**

Card × 50 → List<T>

# Data Model

- Each affirmation is one data class instance: a string resource + an image resource

```
data class Affirmation(
    @StringRes val stringResourceId: Int,
    @DrawableRes val imageResourceId: Int
)
```



# Where the Data Comes From

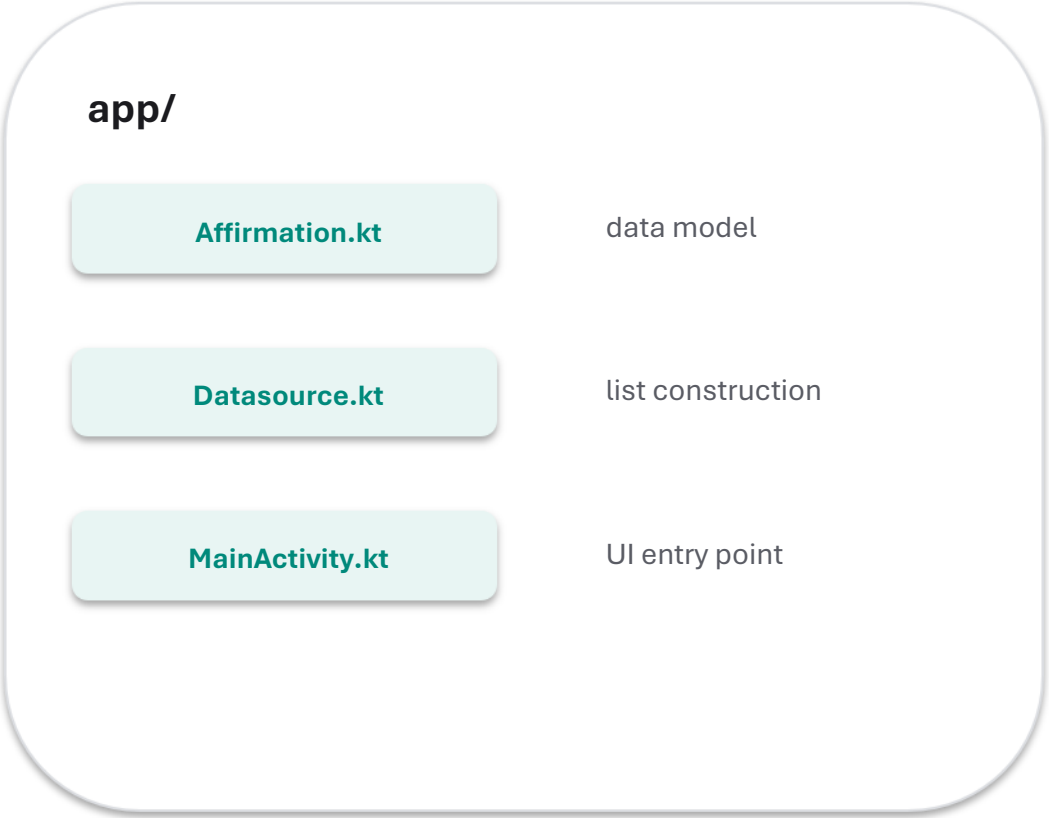
- Datasource builds the full **List<Affirmation>** that the UI will render

```
class Datasource {
    fun loadAffirmations(): List<Affirmation> {
        return listOf(
            Affirmation(R.string.affirmation1, R.drawable.image1),
            Affirmation(R.string.affirmation2, R.drawable.image2),
            // ...
        )
    }
}
```



# Project Structure

- Three files matter today: **Affirmation.kt, Datasource.kt, MainActivity.kt**



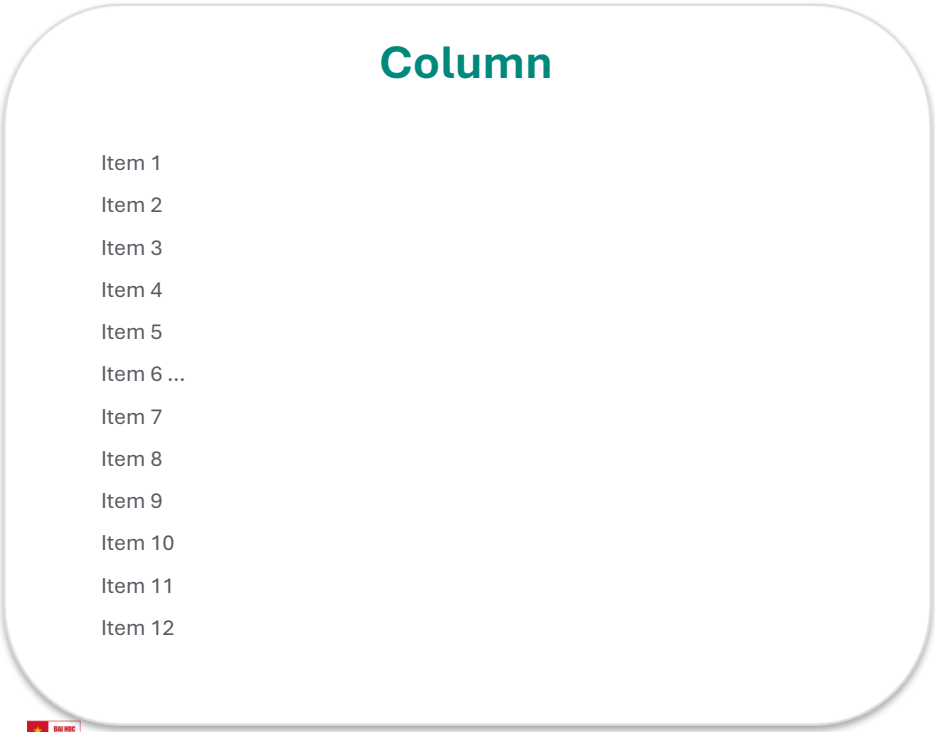
# Data Flow Diagram

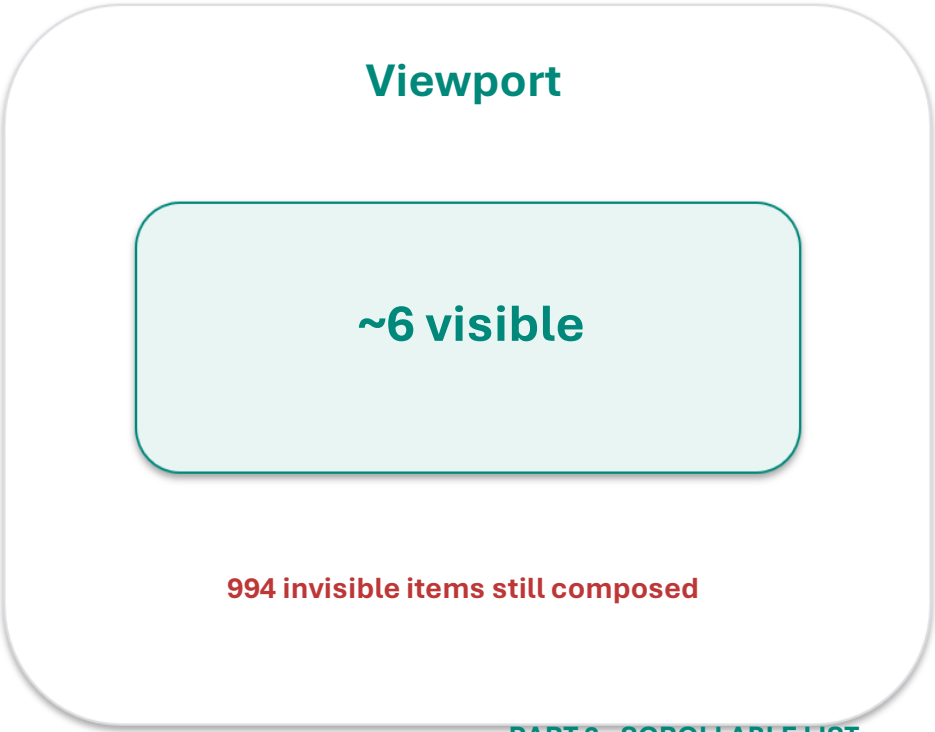
- Data → List → Composable → Screen



# Problem: Column at Scale

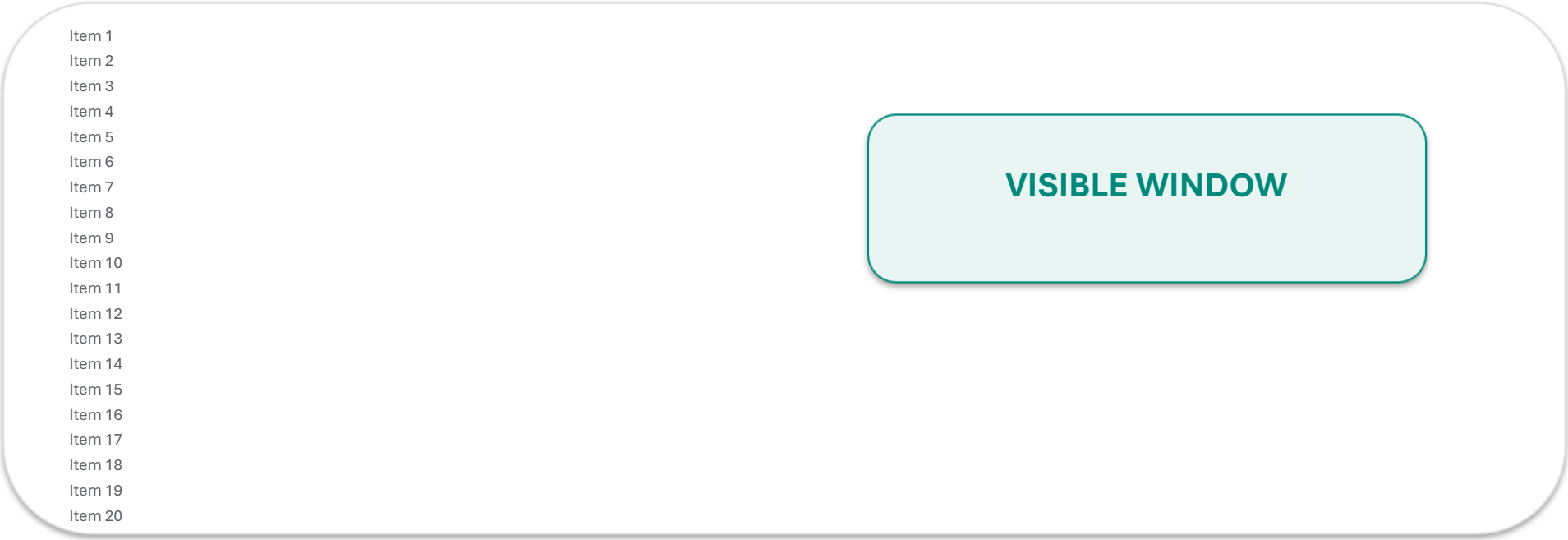
- Column stacks every child vertically, and composes all of them immediately
- A Column with 1,000 items composes and measures all 1,000 — even off-screen



# LazyColumn

- LazyColumn only composes and lays out visible items plus a small buffer



# Column vs. LazyColumn

- Same visual result for a short list; very different work for a long list

| | Column | LazyColumn |
|---|---|---|
| Composes | All children immediately | Only visible children |
| Good for | Small, fixed content | Long / unknown-length lists |
| Views analogue | LinearLayout | RecyclerView |

# Where Does LazyColumn Go?

- Add LazyColumn inside AffirmationList to render the passed list

```
@Composable
private fun AffirmationList(
    affirmationList: List<Affirmation>,
    modifier: Modifier = Modifier
) { }
```

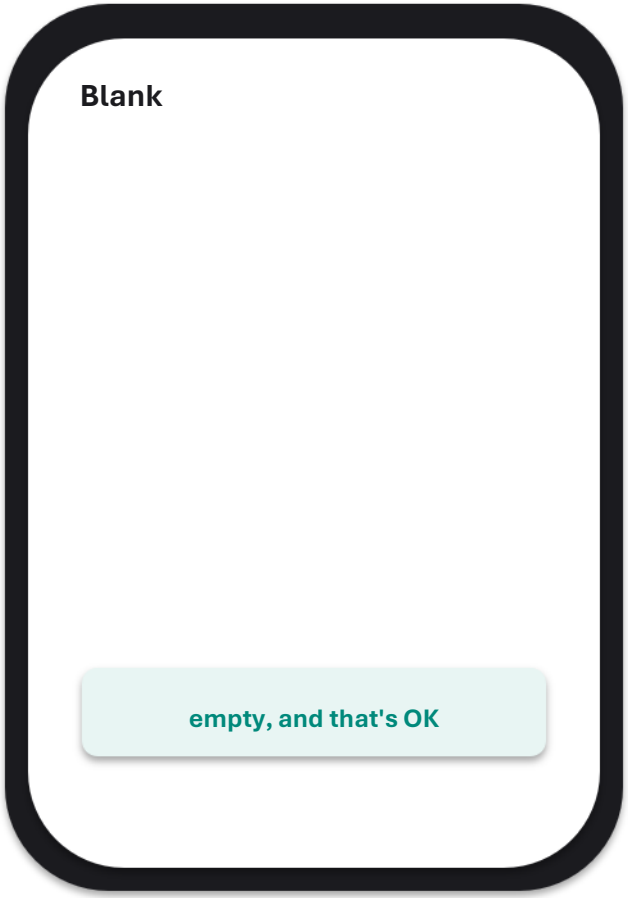
**List<Affirmation>**

already passed in

# Step 1: Declare an Empty LazyColumn

- LazyColumn starts empty — just like an empty Column

```
@Composable
private fun AffirmationList(
    affirmationList: List<Affirmation>,
    modifier: Modifier = Modifier
) {
    LazyColumn { }
}
```



# Step 2: Introducing items()

- items() tells LazyColumn how many items exist and what data each item holds

```
LazyColumn {
    items(affirmationList) { affirmation ->
        // one item UI goes here
    }
}
```

List<T>

item lambda

# Step 3: Rendering Placeholder Text

- Render each affirmation's text as plain Text before styling

```
items(affirmationList) { affirmation ->
    Text(text = stringResource(affirmation.stringResourceId))
}
```



# Step 4: Extracting AffirmationCard

- Move item UI into its own composable so the list only manages the list

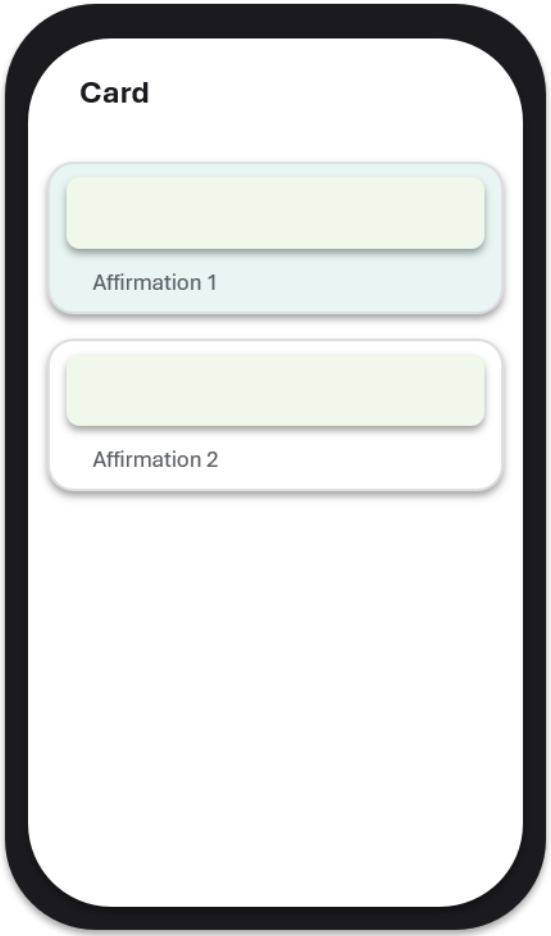
```
@Composable
fun AffirmationCard(
    affirmation: Affirmation,
    modifier: Modifier = Modifier
) {
    Card(modifier = modifier) {
        // image + text
    }
}
```



# Step 5: Image + Text Inside the Card

- Column inside Card stacks image above text

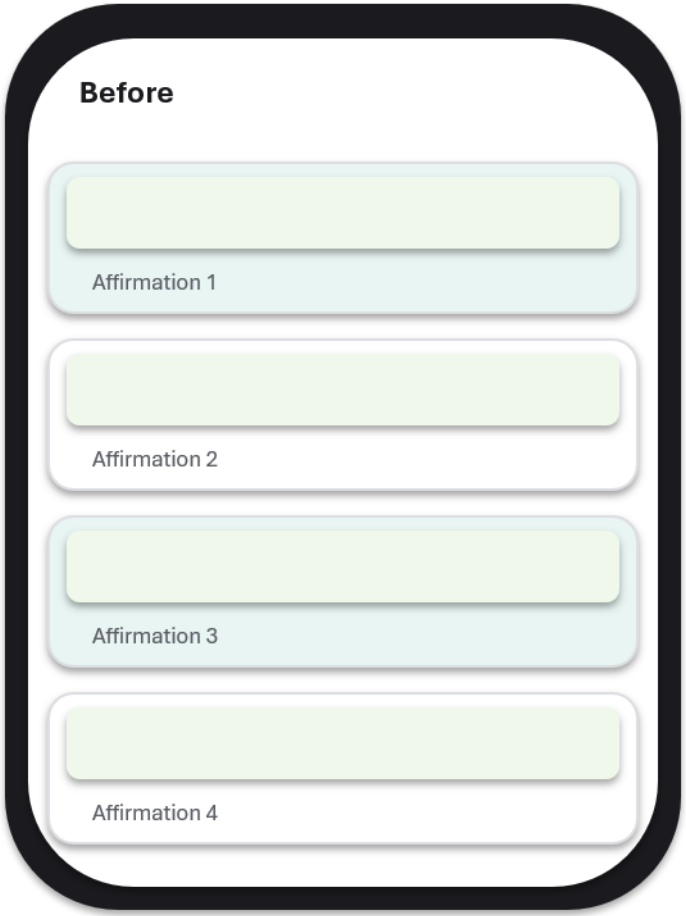
```
Card(modifier = modifier) {
    Column {
        Image(
            painter = painterResource(affirmation.imageResourceId),
            contentDescription = null,
            modifier = Modifier
                .fillMaxWidth()
                .height(194.dp),
            contentScale = ContentScale.Crop
        )
        Text(
            text = stringResource(affirmation.stringResourceId),
            style = MaterialTheme.typography.headlineSmall,
            modifier = Modifier.padding(16.dp)
        )
    }
}
```



# Step 6: Wiring the Card Into the List

- items() now calls AffirmationCard instead of raw Text

Plain text









Styled cards

# The Finished AffirmationList

```
@Composable
private fun AffirmationList(
    affirmationList: List<Affirmation>,
    modifier: Modifier = Modifier
) {
    LazyColumn(modifier = modifier) {
        items(affirmationList) { affirmation ->
            AffirmationCard(
                affirmation = affirmation,
                modifier = Modifier.padding(8.dp)
            )
        }
    }
}
```

- container
- items()
- item
- padding

# Layout Spacing: Arrangement

- verticalArrangement controls the gap between items

```
LazyColumn(
    verticalArrangement = Arrangement.spacedBy(8.dp)
) { /* items */ }
```



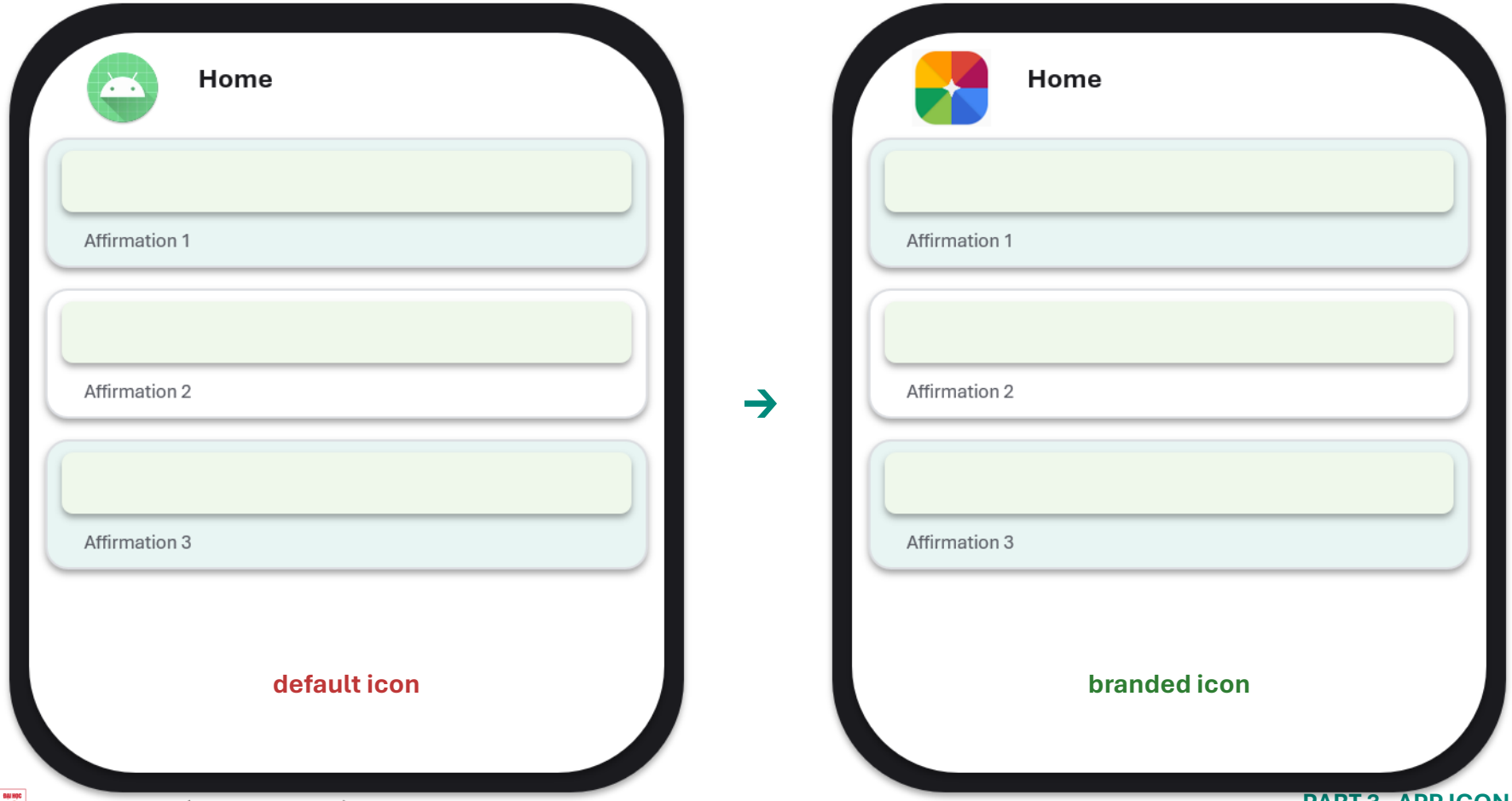
# List Bugs

- Most list bugs come from forgetting items() or misplacing Modifier

| Wrong | Why | Correct |
| --- | --- | --- |
| LazyColumn { Card(a) } | No items() | items(list) { Card(it) } |
| Card(list[0]) | Ignores current item | Use lambda item |
| Padding on list | Pads whole list | Pad each card / contentPadding |

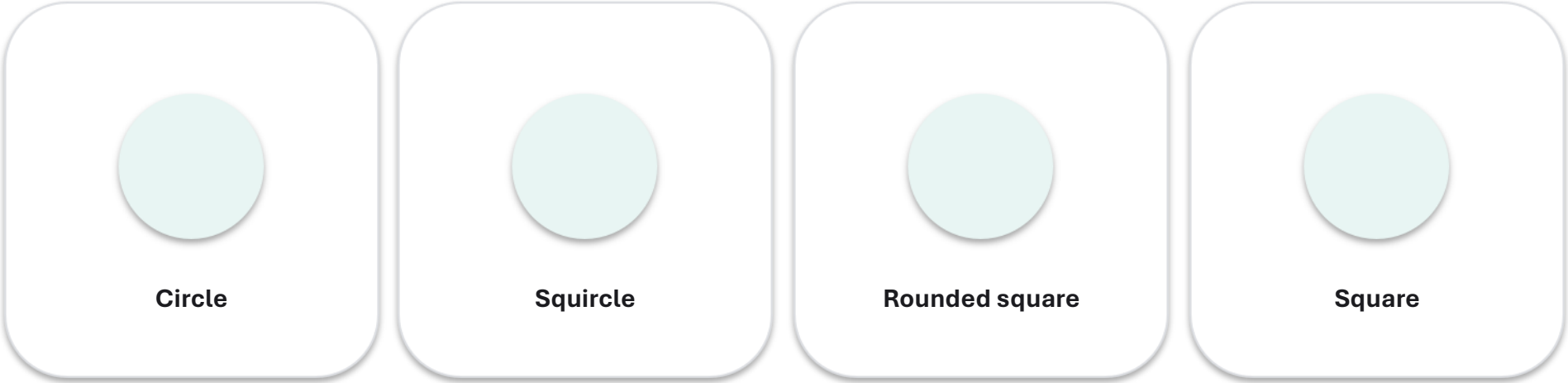
# Problem: A Generic Icon

- The app works, but the home screen still shows the default template icon

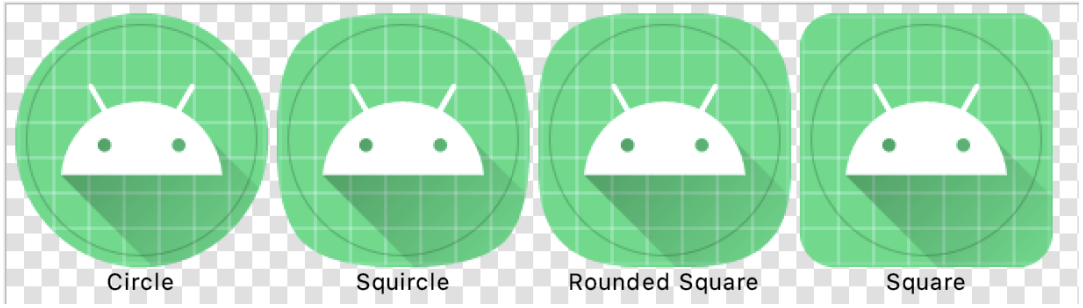


# What Makes Android Icons Special

- The phone maker's launcher can decide the final icon shape

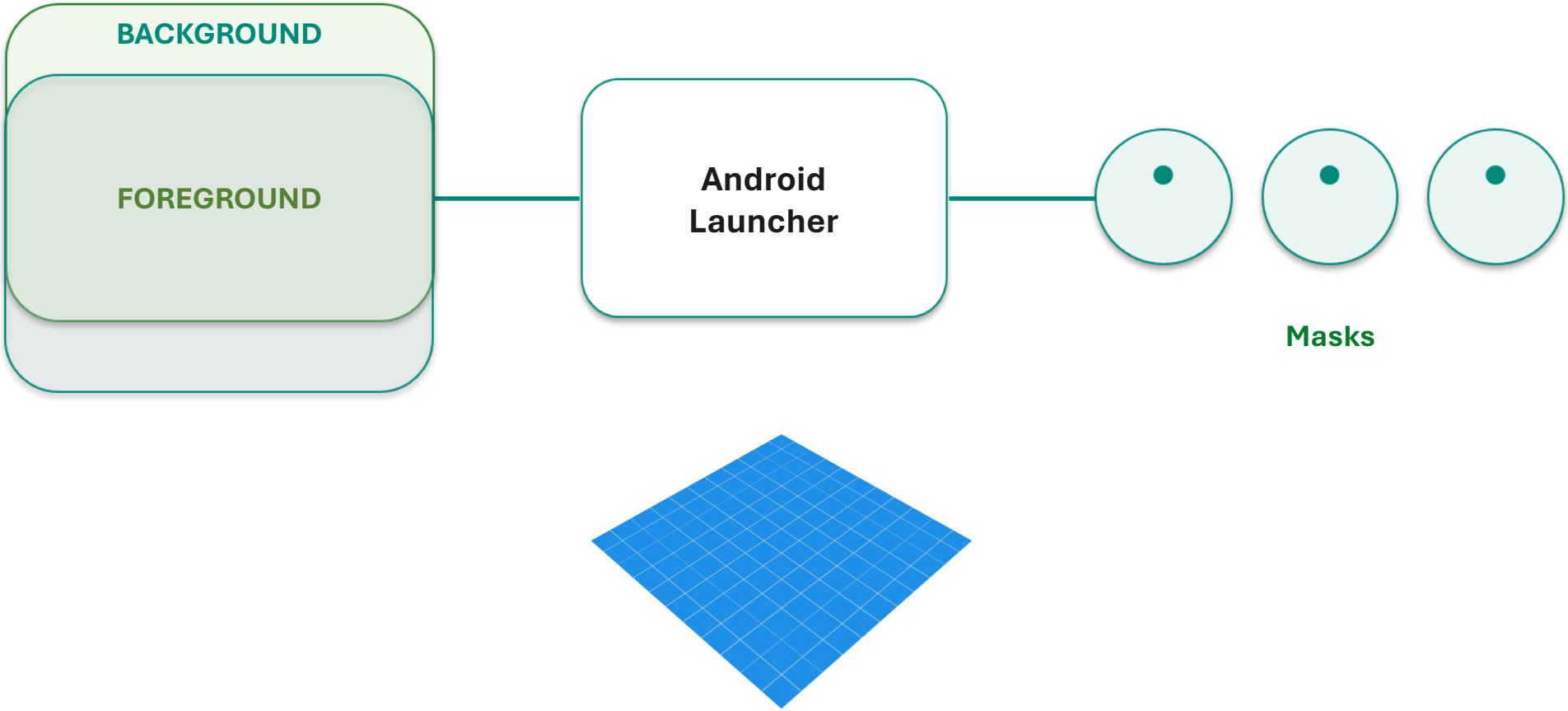



**Same icon image, different masks**



# Adaptive Icon

- Adaptive icon = foreground + background; Android combines and masks them at runtime



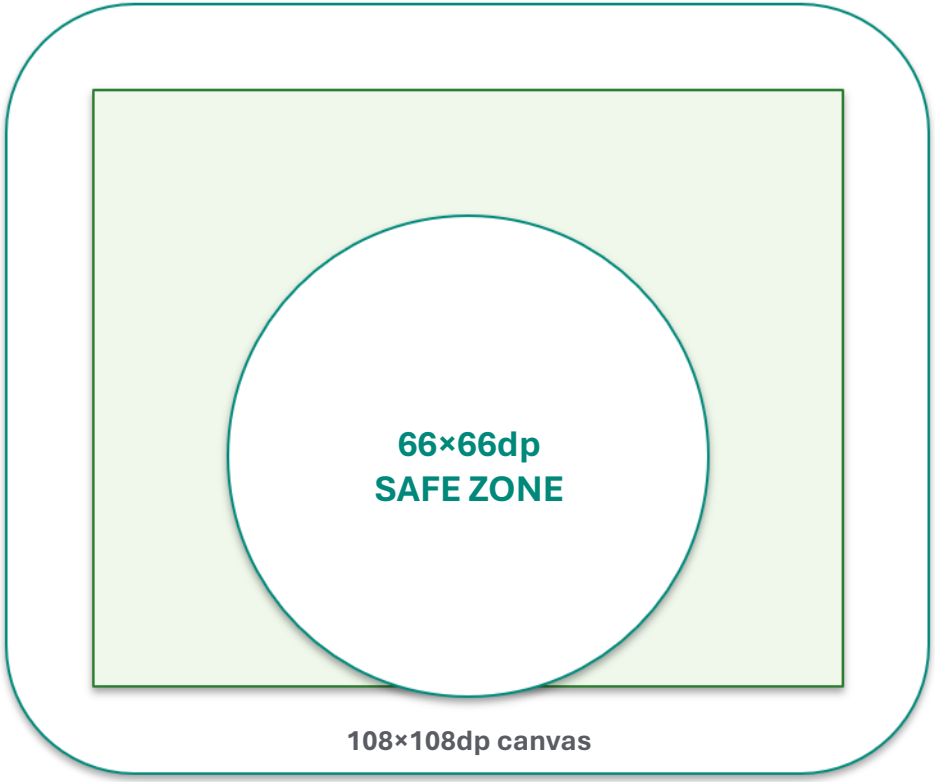
# Foreground vs Background

- Foreground is the logo/symbol; background is a solid color or simple pattern



# The Safe Zone

- A 108×108dp canvas and an inner 66×66dp safe zone; verify the exact current figure before delivery




[VERIFY before delivery]

Figure:
108×108dp canvas
66×66dp guaranteed-safe inner zone

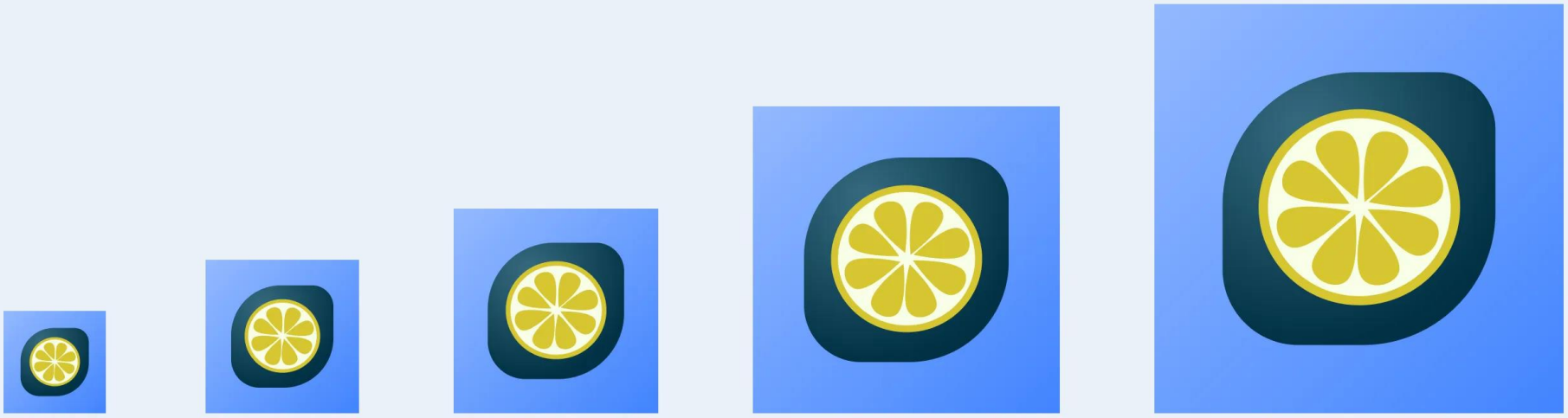
# Where Icons Live: mipmap

- mipmap is the conventional home for launcher/app icon resources

| Folder | Typical contents |
| --- | --- |
| res/drawable/ | In-app images; vector icons |
| res/mipmap/ | Launcher / app icon resources |

# Why So Many Density Folders?

- Android ships multiple resolutions because screens pack pixels differently



1 mdpi 80x80px

2 hdpi 120x120px

3 xhdpi 160x160px

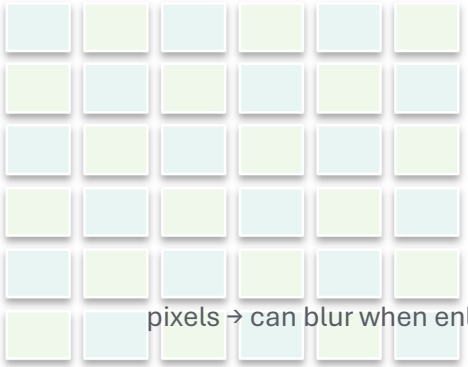
4 xxhdpi 240x240px

5 xxxhdpi 320x320px

# Vector Drawable: A Different Kind of Image

- Vector drawable stores shapes/math rather than pixels and scales cleanly

**BITMAP**



pixels → can blur when enlarged

**VECTOR DRAWABLE**




paths + coordinates → scales cleanly

# Vector Drawable in Practice

- A vector drawable is XML describing paths, not an embedded photo

```
<vector xmlns:android="…"
android:width="24dp" android:height="24dp"
android:viewportWidth="24" android:viewportHeight="24">
  <path
        android:fillColor="#FFFFFF"
        android:pathData="M12,2L2,7v10l10,5 10,-5V7z" />
</vector>
```

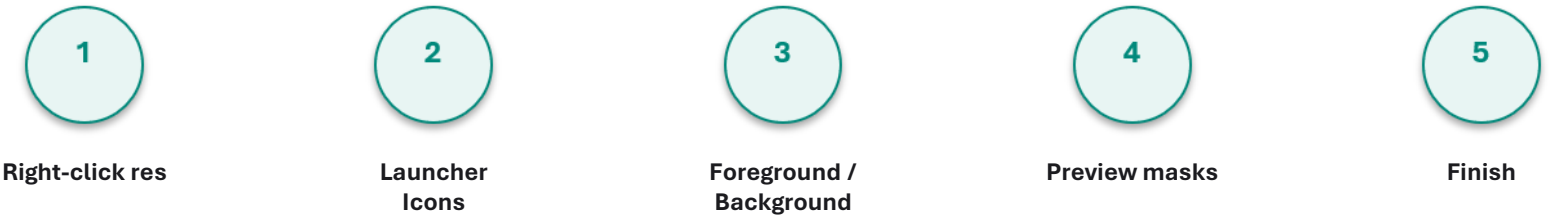


VECTOR

# Using Image Asset Studio

- Image Asset Studio builds density variants and both layers from source art

**Image Asset Studio · annotated workflow**



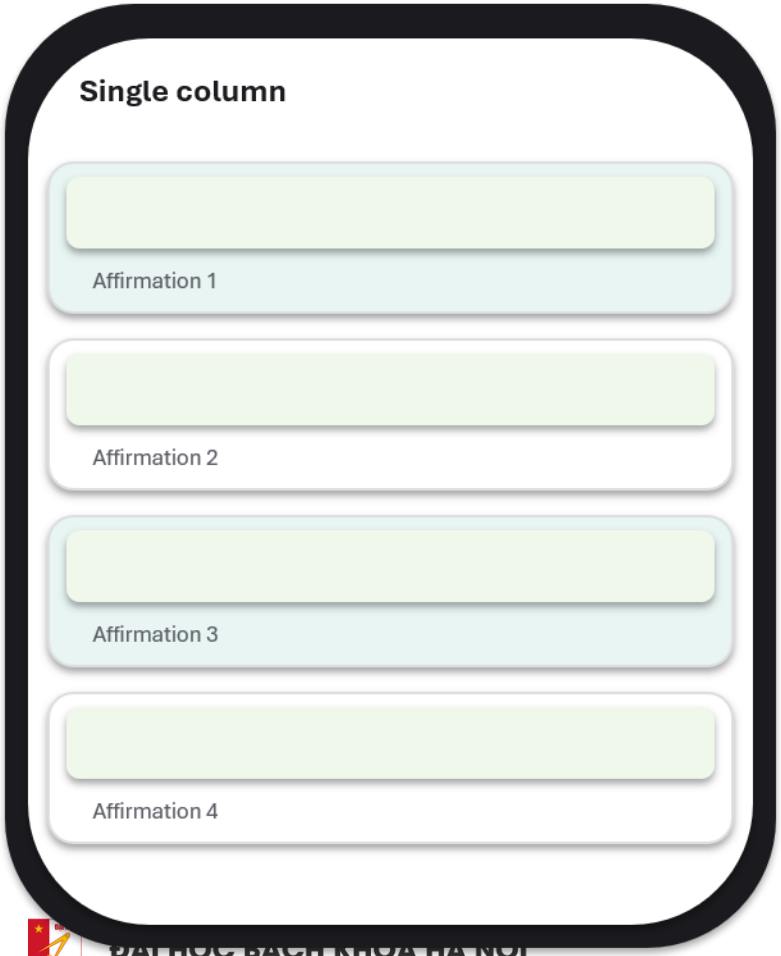
# Icon Errors

- Common errors: edge clipping, foreground / background confusion, manual PNG placement

| Wrong | Why | Correct |
|---|---|---|
| Edge detail | Can be clipped | Keep key content inside safe zone |
| Wrong layer source | Foreground/background confusion | Double-check each layer |
| Manual PNG / missing items() | Skips tooling or DSL | Use Image Asset Studio / items() |

# Problem: Wasted Horizontal Space

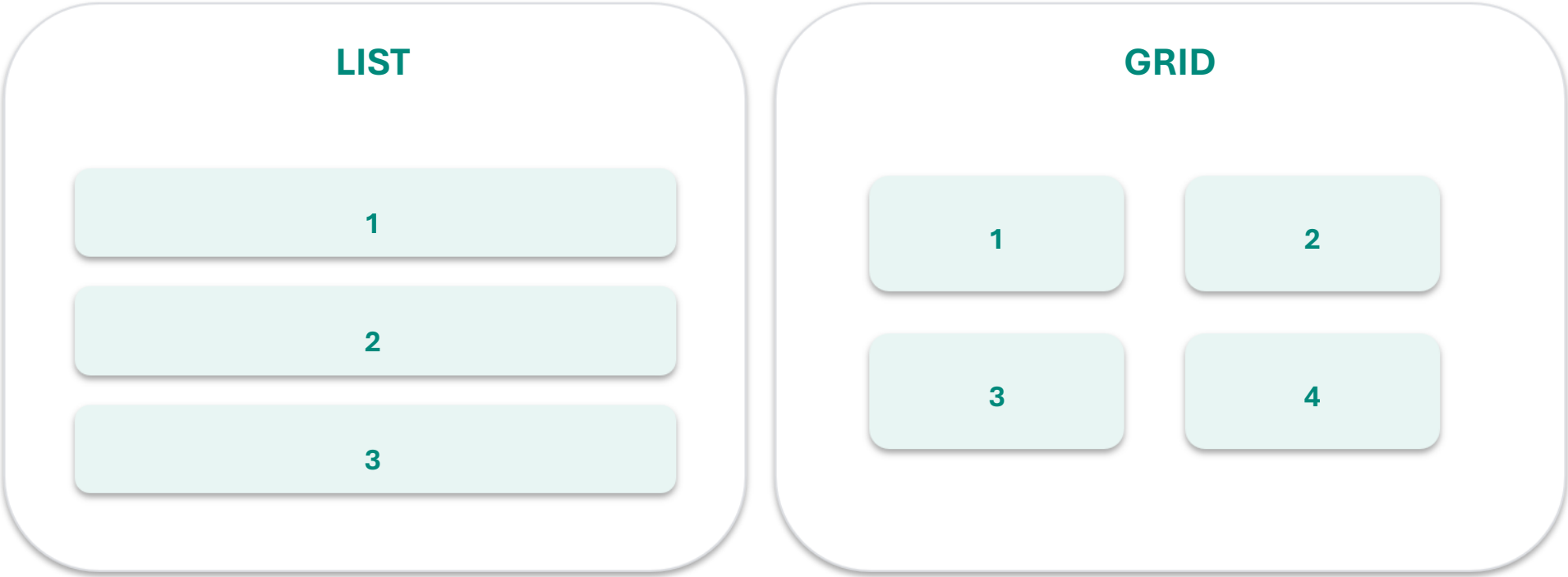
- A single-column list can waste width when items are small or naturally square




EMPTY HORIZONTAL SPACE

# List vs Grid

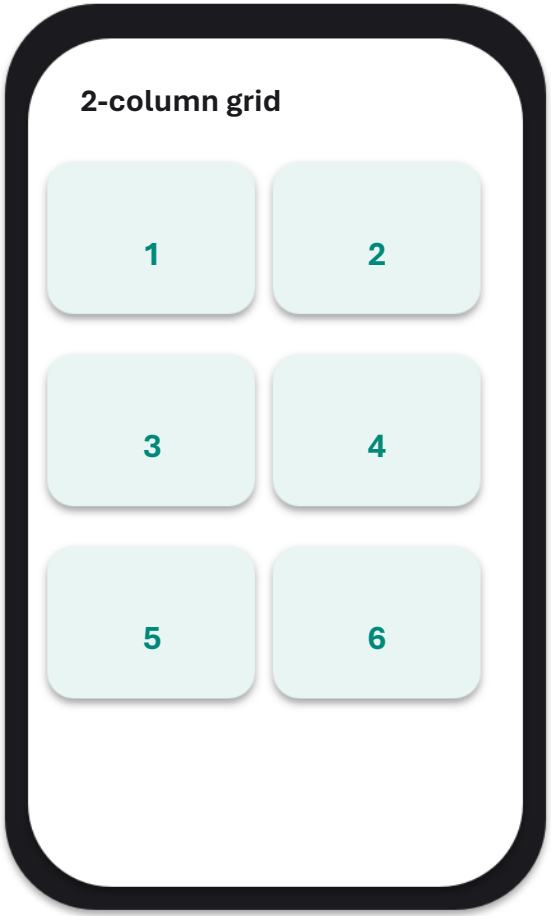
- Grid is a different shape for a different content need, not a replacement for lists



# From LazyColumn to LazyVerticalGrid

- LazyVerticalGrid works like LazyColumn plus one new parameter: columns

```
LazyVerticalGrid(
    columns = GridCells.Fixed(2)
) {
    items(topicList) { topic ->
        TopicCard(topic)
    }
}
```



# Fixed vs Adaptive Columns

- Fixed sets an exact count; Adaptive fits as many columns as the minimum width allows

```
GridCells.Fixed(2)
// exactly 2 columns
```

```
GridCells.Adaptive(
    minSize = 128.dp
)
// as many as fit
```

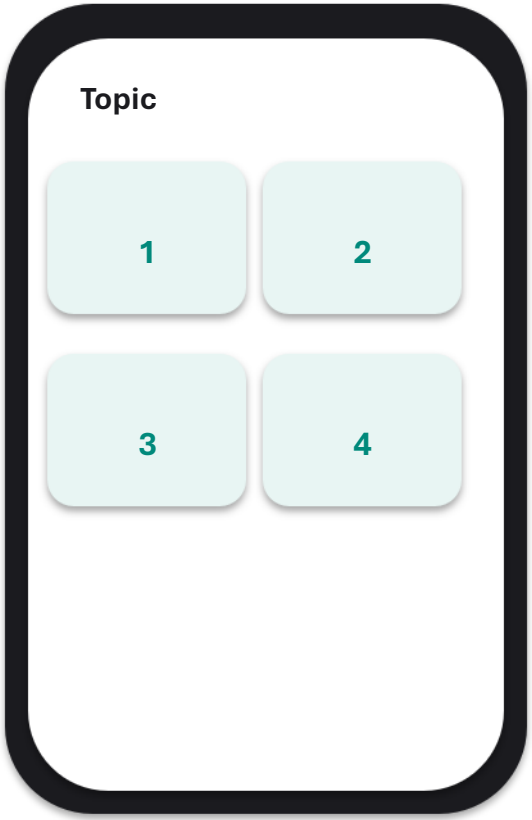





# Building the Grid Item

- Like AffirmationCard, give the grid a focused item composable

```
@Composable
fun TopicCard(topic: Topic, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
        Box {
            Image(painter = painterResource(topic.imageRes), contentDescription
= null)
            Text(text = stringResource(topic.stringRes))
        }
    }
}
```



# Grid Common Mistakes

- Use the lazy grid API, choose realistic adaptive minSize, and keep items() inside the grid

| Wrong | Why | Correct |
|---|---|---|
| Manual Row in LazyColumn | Loses proper lazy grid shape | Use LazyVerticalGrid |
| Wrong layer source | Foreground/background confusion | Double-check each layer |
| Adaptive 500dp / missing items() | Poor phone fit / wrong DSL | Choose realistic minSize / use items() |

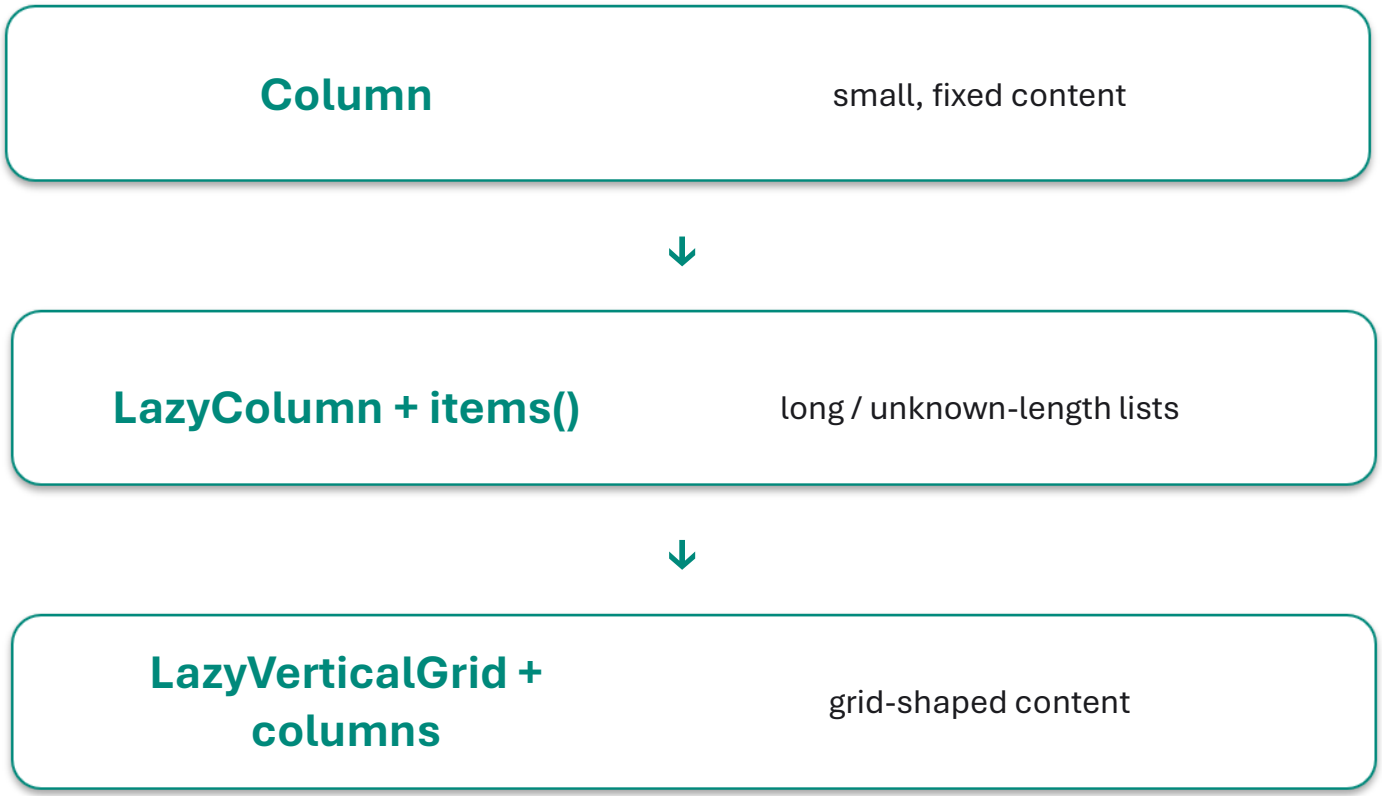
# Summary

- Seven core ideas connect the list, icon, resource, vector, and grid topics

1. Column vs LazyColumn
2. items()
3. Reusable cards
4. Adaptive icon layers
5. mipmap
6. Vector drawables
7. LazyVerticalGrid

# Summary

- Choose the container based on scale and content shape



HUST

hust.edu.vn fb.com/dhbkhn

# THANK YOU !